

Основные режимы функционирования блочных шифров

В предыдущем разделе были даны примеры реальных блочных шифров. Теперь мы можем думать о (идеализированном) блочном шифре, как о преобразовании входного блока X в выходной блок Y с участием секретного ключа K ,

$$Y = E_K(X),$$

причем это преобразование должно иметь следующие свойства:

- 1) при известном Y , но неизвестном K практически невозможно узнать X ;
- 2) при произвольных известных X и Y , но неизвестном K практически невозможно узнать K .

Виды атак

- **По шифротексту.**
- **По известному тексту.** Это происходит, если Ева получает в свое распоряжение какие-либо открытые тексты, соответствующие ранее переданным зашифрованным. Сопоставляя пары «текст–шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту.** Противник пользуется не только предоставленными ему парами «текст–шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

Режим ECB (Electronic CodeBook)

Сообщение X разбивается на блоки $X = X_1, X_2, \dots, X_t$. Каждый блок шифруется блоковым шифром

$$Y_i = E_K(X_i), \quad 1 \leq i \leq t.$$

Получаем зашифрованное сообщение $Y = Y_1, Y_2, \dots, Y_t$. Дешифрование выполняется по правилу

$$X_i = E_K^{-1}(Y_i), \quad 1 \leq i \leq t.$$

Дешифрование сообщения можно производить, выбирая блоки шифротекста в произвольном порядке. Такой режим удобен во многих реальных ситуациях.

Например, можно работать с базой данных, хранящейся в зашифрованном виде. Однако при таком использовании одинаковые записи будут зашифрованы одинаково.

Говорят, что в режиме ЕСВ шифр сохраняет «образ данных», т.е. некий «рисунок» или шаблон, характерный для данных. Это может дать некоторую информацию противнику.

Например, если количество различных записей в базе данных невелико (что нередко случается), то противник может составить словарь шифротекстов и вскрыть базу на основе частотного анализа (ему в этом случае не понадобится вскрывать сам шифр).

Рекомендуется использовать режим **ЕСВ** только в случаях, когда размер отдельного элемента данных в сообщении, к которому требуется осуществлять непосредственный (прямой) доступ, меньше размера блока. Остаток блока, свободный от данных, рекомендуется заполнять случайно выбираемыми битами. Тогда даже одинаковые элементы данных будут иметь разные шифротексты. При дешифровании биты заполнения просто отбрасываются.

Режим CBC (Cipher-Block Chaining)

Зашифрованное сообщение получается по следующему правилу:

$$Y_i = E_K(X_i \oplus Y_{i-1}), \quad 1 \leq i \leq t,$$

т.е. каждый последующий блок открытого текста предварительно закрывается предыдущим зашифрованным блоком. Слово Y_0 должно быть определено заранее и известно при шифровании и дешифровании. Полученное зашифрованное сообщение можно дешифровать следующим образом:

$$X_i = Y_{i-1} \oplus E_K^{-1}(Y_i), \quad 1 \leq i \leq t.$$

Мы получаем шифротекст, в котором каждый следующий блок зависит от предыдущих. Данный режим разрушает «образ данных».

Потоковые шифры

Шифр Вернама является совершенно секретным, т.е. при его использовании противник, перехвативший зашифрованное сообщение, не получает никакой информации об исходном сообщении.

В шифре Вернама зашифрованное сообщение $y_1, y_2, \dots, y_k$ получается из исходного сообщения $x_1, x_2, \dots, x_k$ и ключа $z_1, z_2, \dots, z_k$ при помощи операции шифрования, задаваемой равенством

$$y_i = x_i \oplus z_i, \quad i = 1, 2, \dots, k. \quad (8.12)$$

Шифр Вернама совершенен только в том случае, если ключ $z_1, z_2, \dots, z_k$ образован из независимых и равновероятных символов и используется только один раз.

Поэтому необходимо генерировать случайные последовательности очень большого объема и передавать их по закрытым каналам связи, что затруднительно.

Поэтому давно была высказана идея использовать в качестве ключа последовательности не случайные, а порожденные при помощи генераторов псевдослучайных чисел. В этом случае в качестве секретного ключа используется начальное значение или состояние генератора.

Однако в этом случае система уже не будет совершенно секретной.

Максимум на что мы можем надеяться, это то, что для раскрытия этой системы потребуется очень много времени (например, нужно будет выполнить перебор всех возможных начальных состояний генератора).

В качестве возмещения за потерю совершенности мы получаем возможность использовать короткие (обычно несколько сотен бит) секретные ключи, которые значительно проще распределять и хранить (а при использовании систем с открытым ключом их можно и вычислять).

Определение 8.1. Шифр, построенный на основе (8.12), где в качестве $z_1, z_2, \dots, z_k$ используется псевдослучайная последовательность, называется *потокowym шифром* (stream cipher).

Как правило, исходное сообщение и ключевая последовательность представляют собой независимые потоки бит. Так как шифрующее (и дешифрующее) преобразование для всех потоковых шифров одно и то же, они различаются только способом построения генераторов псевдослучайных чисел. Действительно, чтобы из шифротекста $y_1, y_2, \dots, y_k$, полученного по формуле (8.12), восстановить сообщение $x_1, x_2, \dots, x_k$, необходимо сгенерировать точно такую же последовательность $z_1, z_2, \dots, z_k$, что и при шифровании, и использовать для дешифрования формулу

$$x_i = y_i \oplus z_i, \quad i = 1, 2, \dots, k. \quad (8.13)$$

Пример 8.1. Один из самых простых генераторов псевдослучайных чисел (линейный конгруэнтный генератор) работает по схеме

$$z_{i+1} = (az_i + b) \bmod c, \quad (8.14)$$

где a , b , c — некоторые константы, а z_{i+1} — очередное псевдослучайное число, вычисляемое по предыдущему z_i . Обязательно задается начальное значение z_0 .

Возьмем в качестве примера $a = 5$, $b = 12$, $c = 23$, и пусть $z_0 = 4$. Вычислим несколько элементов последовательности:

$$z_1 = (4 \cdot 5 + 12) \bmod 23 = 9,$$

$$z_2 = (9 \cdot 5 + 12) \bmod 23 = 11,$$

$$z_3 = (11 \cdot 5 + 12) \bmod 23 = 21,$$

$$z_4 = (21 \cdot 5 + 12) \bmod 23 = 2,$$

$$z_5 = (2 \cdot 5 + 12) \bmod 23 = 22,$$

$$z_6 = (22 \cdot 5 + 12) \bmod 23 = 7,$$

$$z_7 = (7 \cdot 5 + 12) \bmod 23 = 1.$$

Полученная последовательность внешне выглядит как довольно случайная.

Для использования в криптографических целях генератор должен удовлетворять следующим основным требованиям:

- 1) период последовательности должен быть очень большой;
- 2) порождаемая последовательность должна быть «почти» неотличима от действительно случайной, в частности, вычисление числа z_{i+1} по известным предыдущим элементам последовательности без знания ключа должно быть трудной, практически нерешаемой задачей.

Рассмотренный выше линейный конгруэнтный генератор совершенно не годится для криптографических целей, так как известны простые алгоритмы, позволяющие полностью восстановить параметры генератора всего по нескольким элементам порождаемой им последовательности.

Режим OFB (Output FeedBack) блочного шифра

В этом режиме блочный шифр на основе секретного ключа K и некоторого инициализирующего вектора Y_0 формирует псевдослучайную последовательность r - битовых чисел $z_1, z_2, \dots, z_k$, которая может использоваться в (8.12) и (8.13) соответственно для шифрования и дешифрования сообщения. Будем считать, как и ранее, что размер блока шифра равен n бит. Псевдослучайная последовательность получается по схеме:

$$Y_i = E_K(Y_{i-1}),$$

$$z_i = r \text{ старших бит } Y_i, \quad 1 \leq i \leq k$$

(здесь $r, 1 \leq r \leq n$, — параметр метода).

При использовании стойкого блочного шифра можно получить криптостойкий генератор, отвечающий приведенным выше требованиям. А именно, средняя длина периода псевдослучайной последовательности (при случайно выбранных K и Y_0) составляет примерно $r2^{n-1}$ бит. Кроме того, псевдослучайная последовательность «непредсказуема» для противника, так как возможность предсказания (вычисления) z_{i+1} на основе $z_1, \dots, z_i$ означала бы нестойкость шифра по отношению к атаке по известному тексту. Предсказание z_{i+1} становится даже более трудной задачей, чем взлом блочного шифра, если $r < n$.

Обратим внимание на одну особенность, характерную для всех потоковых шифров. Для шифрования каждого отдельного сообщения необходимо использовать разные K и/или Y_0 . В противном случае несколько сообщений будут шифроваться с помощью одних и тех же последовательностей z , и такой шифр может быть раскрыт. Поясним суть проблемы. Пусть два сообщения $u_1, u_2, \dots, u_k$ и $v_1, v_2, \dots, v_k$ зашифрованы с помощью одной и той же последовательности z . Тогда шифротексты будут иметь вид

$$u_1 \oplus z_1, u_2 \oplus z_2, \dots, u_k \oplus z_k \quad \text{и} \\ v_1 \oplus z_1, v_2 \oplus z_2, \dots, v_k \oplus z_k.$$

Сложим оба шифротекста и, с учетом равенства $z_i \oplus z_i = 0$, получим последовательность

$$u_1 \oplus v_1, u_2 \oplus v_2, \dots, u_k \oplus v_k.$$

Мы получили аналог так называемого «шифра с бегущим ключом», когда один текст шифруется с помощью другого текста, взятого из определенного места определенной книги. Известно, что такой шифр не стоек, хотя использовался в эпоху «донаучной» криптографии.

Статистический анализ, основанный на избыточности текстов, позволяет в большинстве случаев достаточно точно восстановить оба сообщения.

Дешифрование сообщений для описанного режима блочного шифра может производиться только с начала, так как невозможно получить произвольный элемент последовательности z , не вычислив предыдущие. В этом смысле режим аналогичен режиму СВС. Преимущество режима OFB заключается в том, что последовательность z может быть сформирована заранее для того, чтобы быстро шифровать или дешифровать сообщения с помощью (8.12), (8.13) в момент их поступления. Это может быть актуально для систем, обрабатывающих данные в реальном масштабе времени.

Режим CTR (CounTeR) блочного шифра

Этот режим очень похож на OFB, но в нем шифруется не предыдущий выход шифра, а просто счетчик, увеличиваемый на каждом шаге на постоянное число (обычно 1). Точнее, схема выглядит следующим образом:

$$z_i = r \text{ старших бит } E_K(Y_0 + i), \quad i = 1, 2, 3, \dots,$$

где r — параметр.

При использовании «идеального» блочного шифра режим CTR обеспечивает те же параметры стойкости, что и OFB. Преимущество режима CTR состоит в том, что любой элемент последовательности z может быть вычислен непосредственно. Это дает возможность шифровать и дешифровать любые фрагменты сообщения независимо друг от друга.

Алгоритм RC4

Алгоритм RC4, предложенный Ривестом в 1994 году, относится к классу алгоритмов, разработанных специально для потоковых шифров. Генераторы псевдослучайных чисел, построенные с помощью таких алгоритмов, как правило, значительно быстрее генераторов, основанных на блоковых шифрах.

Алгоритм RC4 работает с n -битовыми словами (обычно $n = 8$). Все вычисления проводятся по модулю 2^n (остаток $x \bmod 2^n$ вычисляется очень быстро путем выделения n младших бит в x с помощью логической операции «и»). RC4 использует L -словный ключ $K = K_0 K_1 \dots K_{L-1}$ и генерирует последовательность слов $\bar{z} = z_1 z_2 z_3 \dots$, конкретный вид которой определяется ключом K .

Состояние генератора задается таблицей S из 2^n слов и двух переменных i и j . В каждый момент времени таблица S содержит все возможные n -битовые числа в перемешанном виде. Так как каждый элемент таблицы принимает значения в промежутке $[0, 2^n - 1]$, то его можно трактовать двояко: либо как число, либо как номер другого элемента в таблице. Секретный ключ задает только начальное перемешивание чисел в таблице, которое формируется с помощью следующего алгоритма:

$$\begin{aligned}
 &j \leftarrow 0, \quad S \leftarrow (0, 1, \dots, 2^n - 1); \\
 &\text{FOR } i = 0, 1, \dots, 2^n - 1 \text{ DO} \\
 &\quad j \leftarrow (j + S_i + K_{i \bmod L}) \bmod 2^n, \\
 &\quad S_j \leftrightarrow S_i; \\
 &i \leftarrow 0, \quad j \leftarrow 0.
 \end{aligned}$$

После этого генератор готов к работе. Генерация очередного псевдослучайного слова z_i осуществляется следующим образом:

$$\begin{aligned}i &\leftarrow (i + 1) \bmod 2^n; \\j &\leftarrow (j + S_i) \bmod 2^n; \\S_j &\leftrightarrow S_i; \\t &\leftarrow (S_i + S_j) \bmod 2^n; \\z_i &\leftarrow S_t.\end{aligned}$$

Пример 8.2. Пусть $n = 3$, $K = 25$ ($L = 2$).

Сформируем начальную перестановку чисел в таблице S (все вычисления проводим по модулю 8):

	$j = 0,$	$S = (0, 1, 2, 3, 4, 5, 6, 7),$
$i = 0,$	$j = 0 + 0 + 2 = 2,$	$S = (2, 1, 0, 3, 4, 5, 6, 7),$
$i = 1,$	$j = 2 + 1 + 5 = 0,$	$S = (1, 2, 0, 3, 4, 5, 6, 7),$
$i = 2,$	$j = 0 + 0 + 2 = 2,$	$S = (1, 2, 0, 3, 4, 5, 6, 7),$
$i = 3,$	$j = 2 + 3 + 5 = 2,$	$S = (1, 2, 3, 0, 4, 5, 6, 7),$
$i = 4,$	$j = 2 + 4 + 2 = 0,$	$S = (4, 2, 3, 0, 1, 5, 6, 7),$
$i = 5,$	$j = 0 + 5 + 5 = 2,$	$S = (4, 2, 5, 0, 1, 3, 6, 7),$
$i = 6,$	$j = 2 + 6 + 2 = 2,$	$S = (4, 2, 6, 0, 1, 3, 5, 7),$
$i = 7,$	$j = 2 + 7 + 5 = 6,$	$S = (4, 2, 6, 0, 1, 3, 7, 5).$

Теперь вычислим несколько первых элементов псевдослучайной последовательности $\bar{z}$:

$$\begin{aligned} i = 1, j = 0 + 2 = 2, S = (4, 6, 2, 0, 1, 3, 7, 5), t = 2 + 6 = 0, z_1 = 4, \\ i = 2, j = 2 + 2 = 4, S = (4, 6, 1, 0, 2, 3, 7, 5), t = 1 + 2 = 3, z_2 = 0, \\ i = 3, j = 4 + 0 = 4, S = (4, 6, 1, 2, 0, 3, 7, 5), t = 2 + 0 = 2, z_3 = 1, \\ i = 4, j = 4 + 0 = 4, S = (4, 6, 1, 2, 0, 3, 7, 5), t = 0 + 0 = 0, z_4 = 4, \\ i = 5, j = 4 + 3 = 7, S = (4, 6, 1, 2, 0, 5, 7, 3), t = 5 + 3 = 0, z_5 = 4, \\ i = 6, j = 7 + 7 = 6, S = (4, 6, 1, 2, 0, 5, 7, 3), t = 7 + 7 = 6, z_6 = 7 \end{aligned}$$

и т.д. Чтобы воспользоваться формулой (8.12) для получения шифра, числа z_i записываем в двоичном виде. В рассмотренном примере каждое число z_i представляется тремя битами, и мы получаем последовательность

$$\bar{z} = 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ \dots$$